

CMP5354
Software Design
UML Class Diagrams

+Notations: ArrayList<String>
+ModellingActivities:List<String>

+setAuthor(String author)

Re-capping Module Content

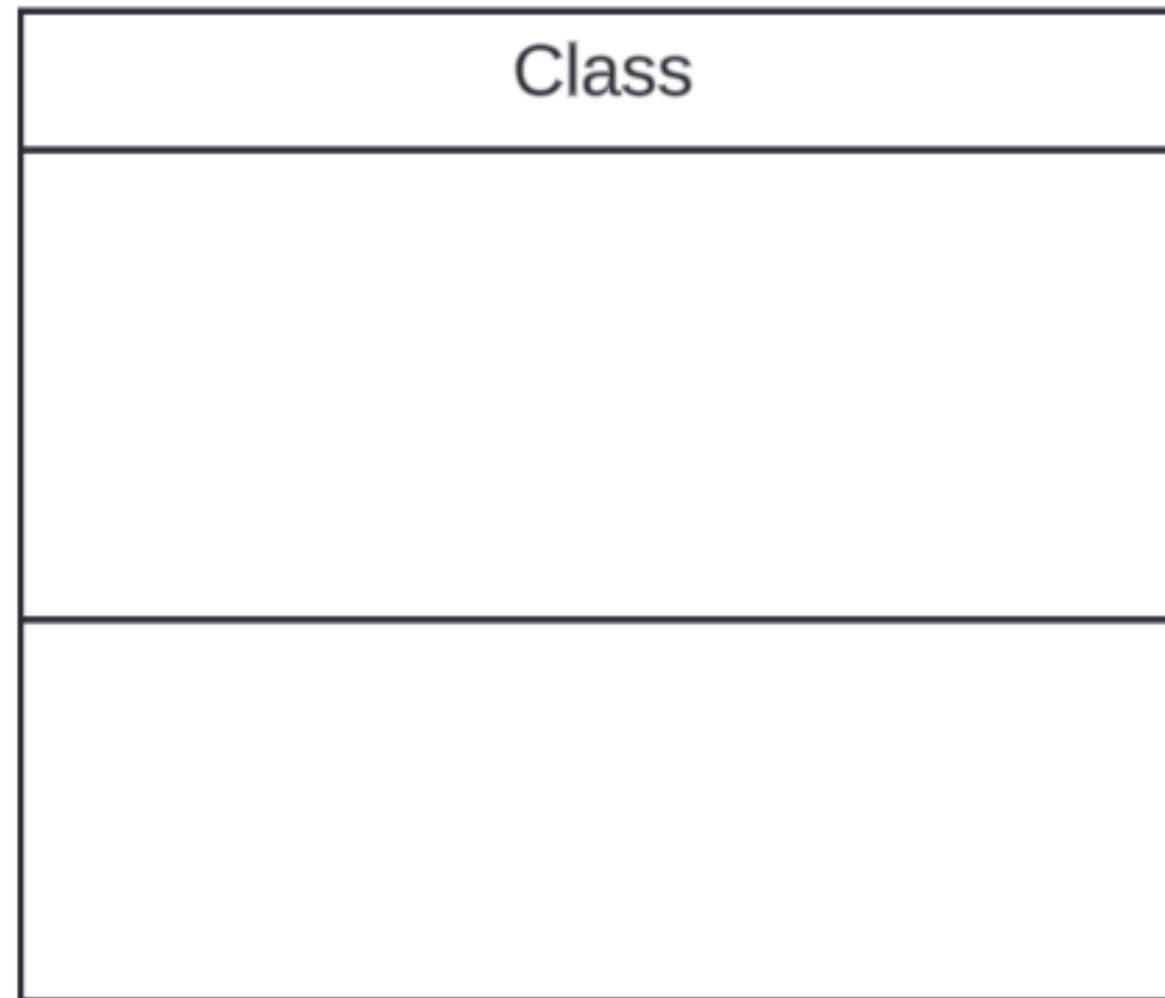
<u>Activity</u>	<u>Definition</u>
Inception	Defining project problem, high-level features, feasibility, target users and solution scope
Requirements Engineering	Eliciting, analyzing, specifying, and validating the functional and non-functional requirements of the solution
Design	Defining solution architecture and components to meet the requirements and guide implementation
Implementation	Translating the design into executable code
Testing	Verifying that the solution meets the functional and non-functional requirements
Maintenance and Evolution	Ongoing process of updating, modifying, and improving a software solution after deployment to address issues and adapt to changing needs

UMLClassDiagramDefinition

A UML diagram that shows the entities to be implemented in the system and the relationships between these entities.

UML Class Diagram Notations

A Class



Classes are a **user-defined blueprint** for an **entity**. It maps out the characteristics, methods, or qualities common to all objects of a certain kind. It will map to a single implemented class (e.g., a single Python file).

Multiple **instances/objects of the same class** can be created (by calling the same constructor/_init_ function multiple times).

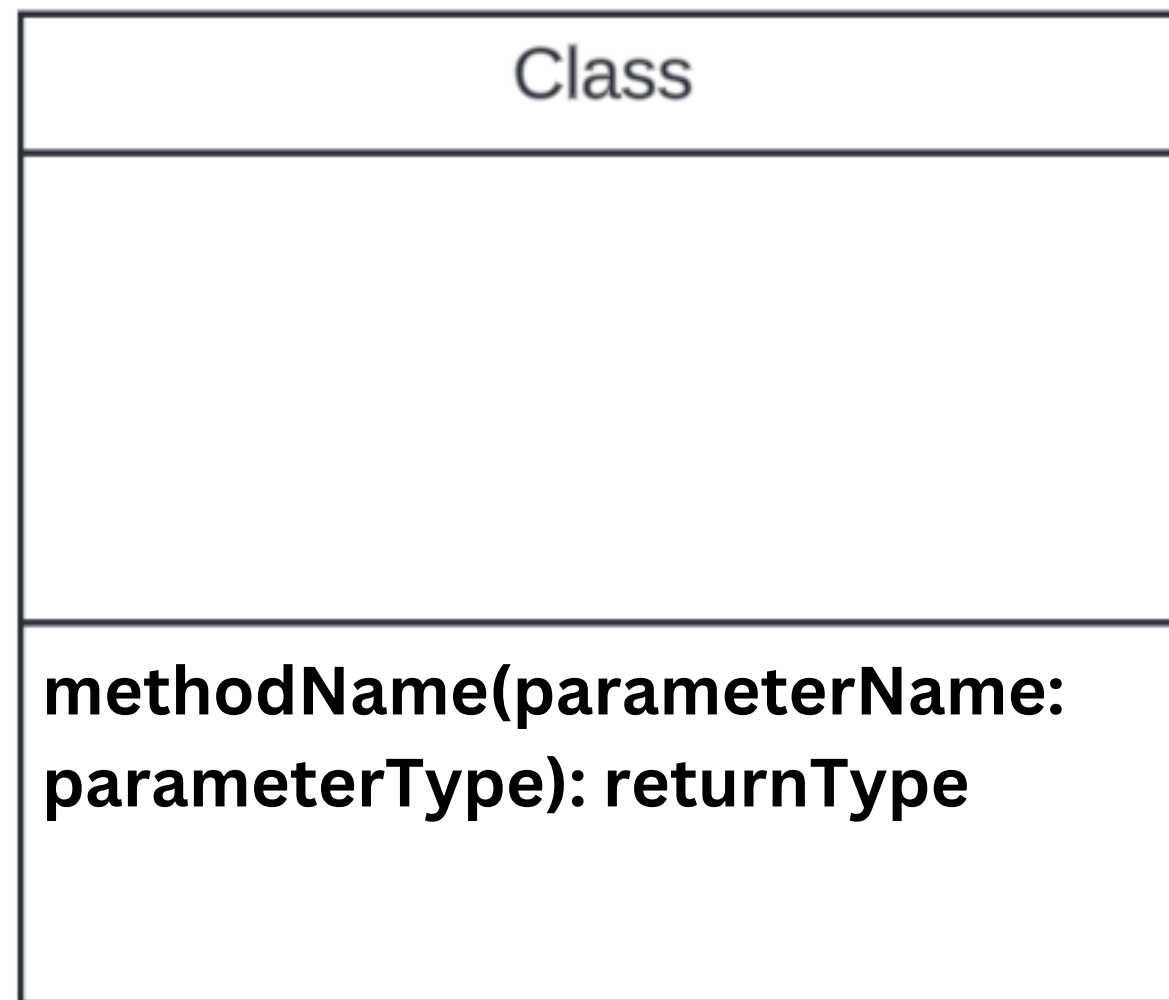
Attribute

Class
attributeName:dataType



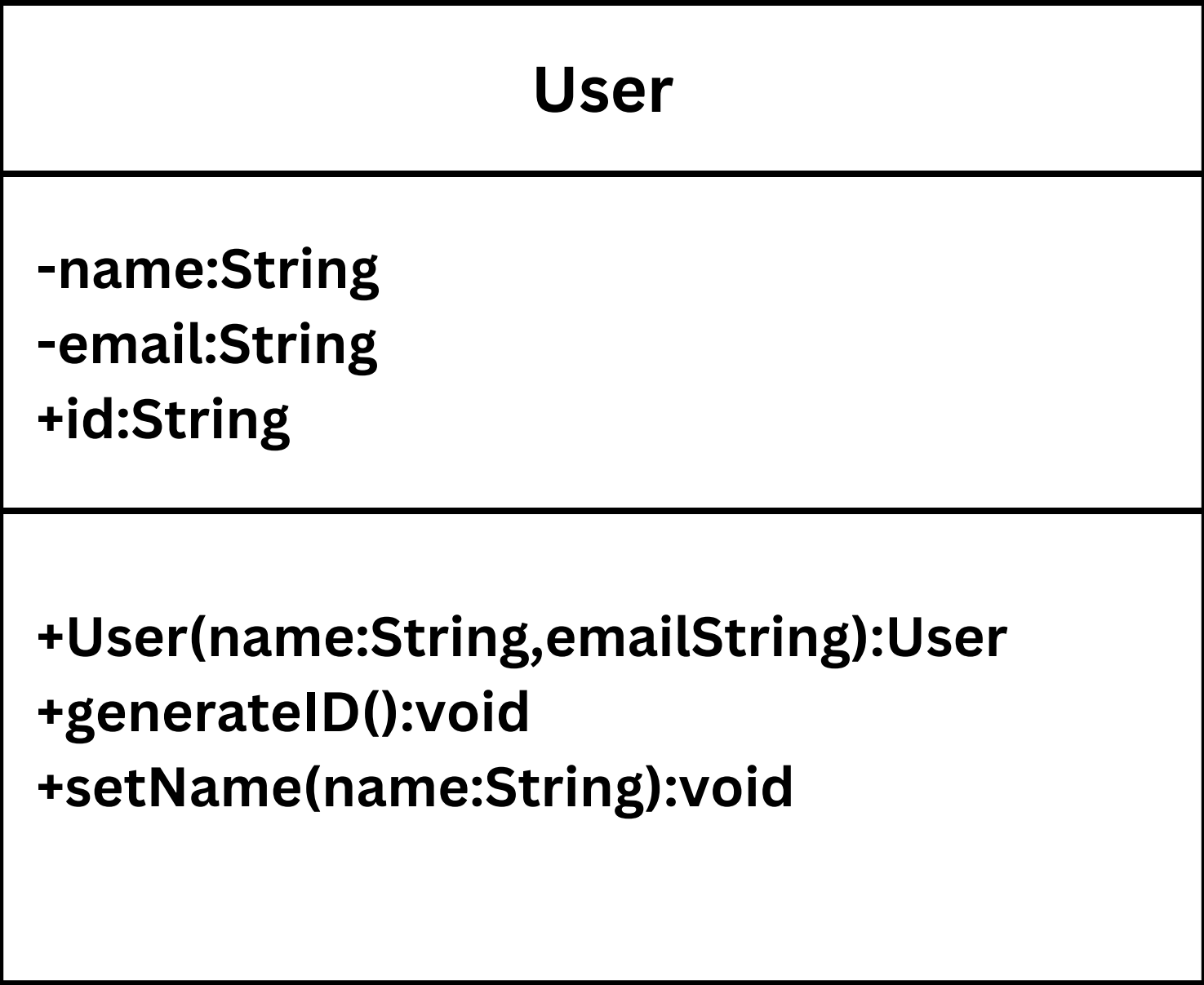
A significant piece of data containing values that describe each instance of that class.

Methods



Operations or Functions

Allow you to specify any behavioral feature of a class.



Visibility

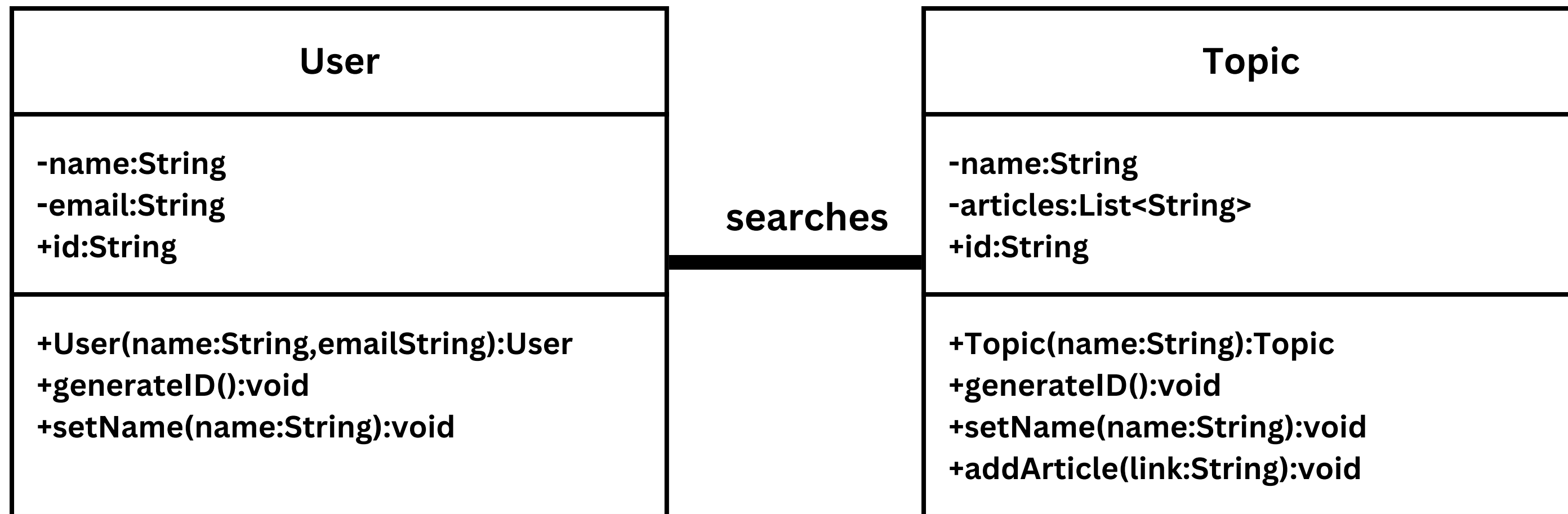
+	Public	All classes can view the information.
-	Private	Information is hidden from all classes outside the partition.
#	Protected	Child class can access the parent class inherited information.

Access Rights

Access Right	public (+)	private (-)	protected (#)
Members of the same class	Yes	Yes	Yes
Members of derived classes	Yes	No	Yes
Members of any other class	Yes	No	No

Relationships

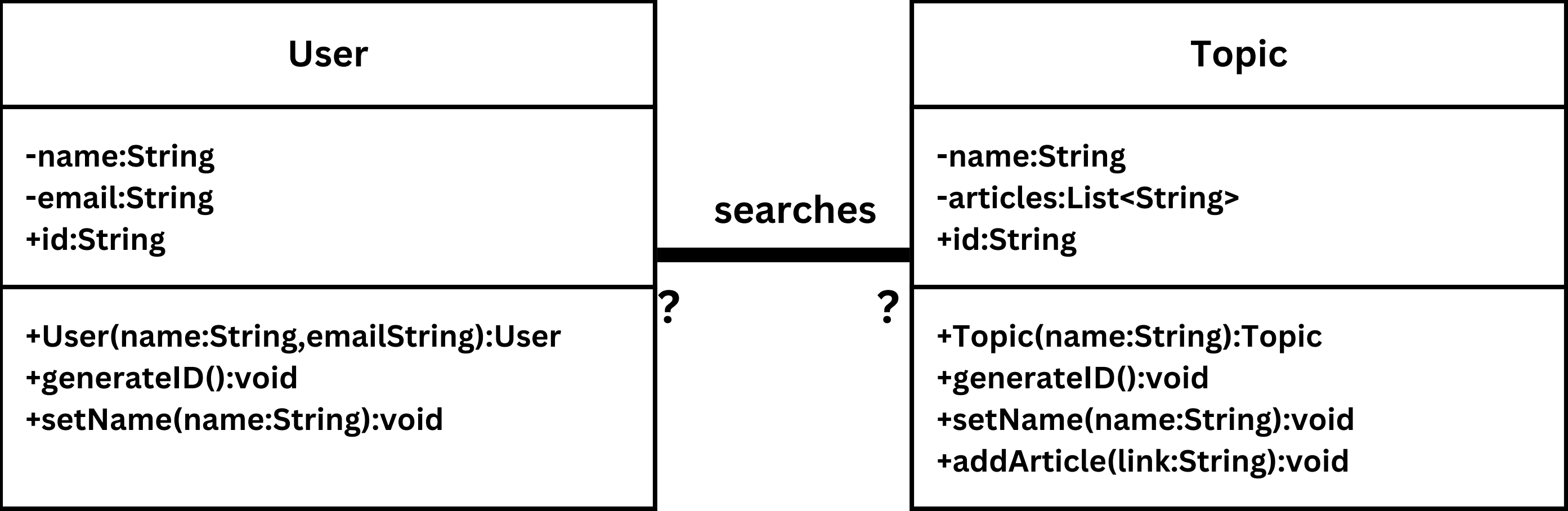
	Association	A labelled relationship or connection between classes
---	--------------------	---

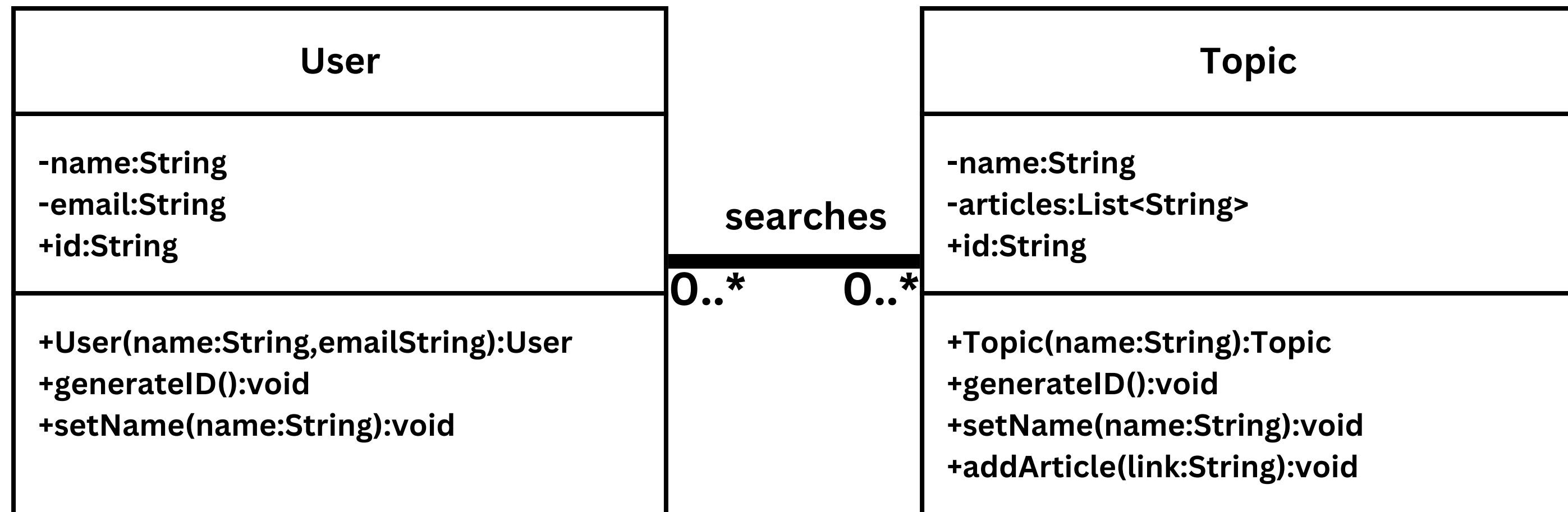


Any logical connection or relationship between classes. For example, users search for topics. In other words, every time a User object is created, it can search for Topic objects created.

Multiplicity

0..1	Zero to One
n	Specific Number
0..*	Zero to Many
1..*	One to Many

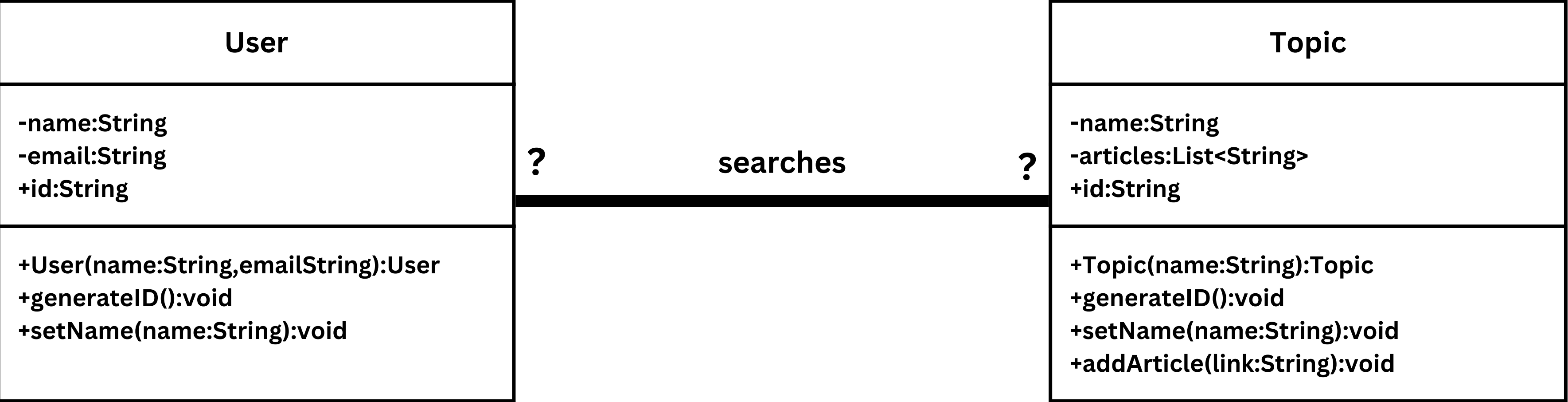


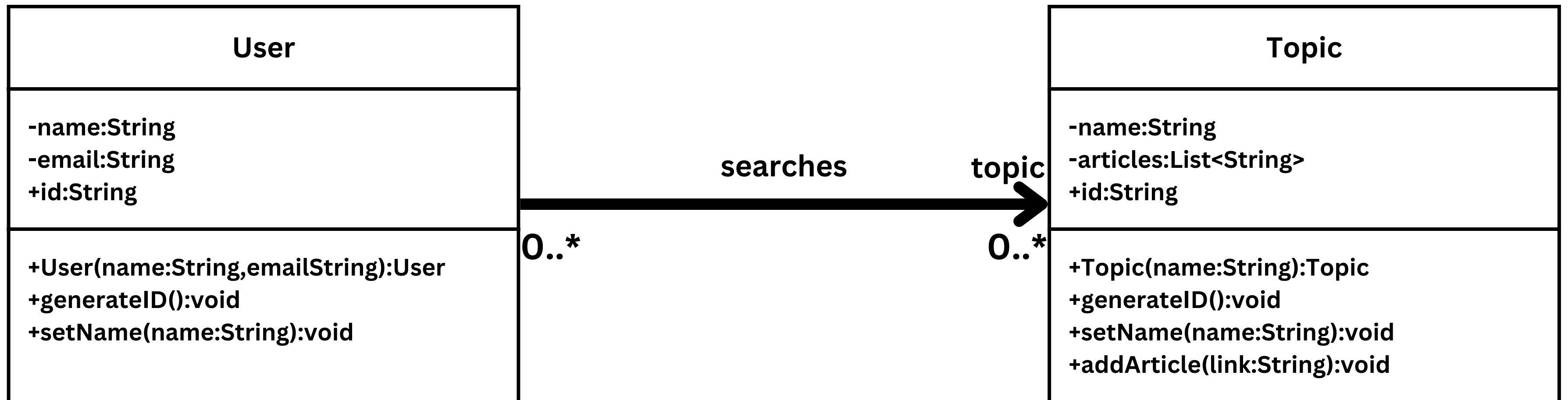


A user can search for no topic or many topics at any point in time..
A topic can be searched for by no users or many users at any point in time.

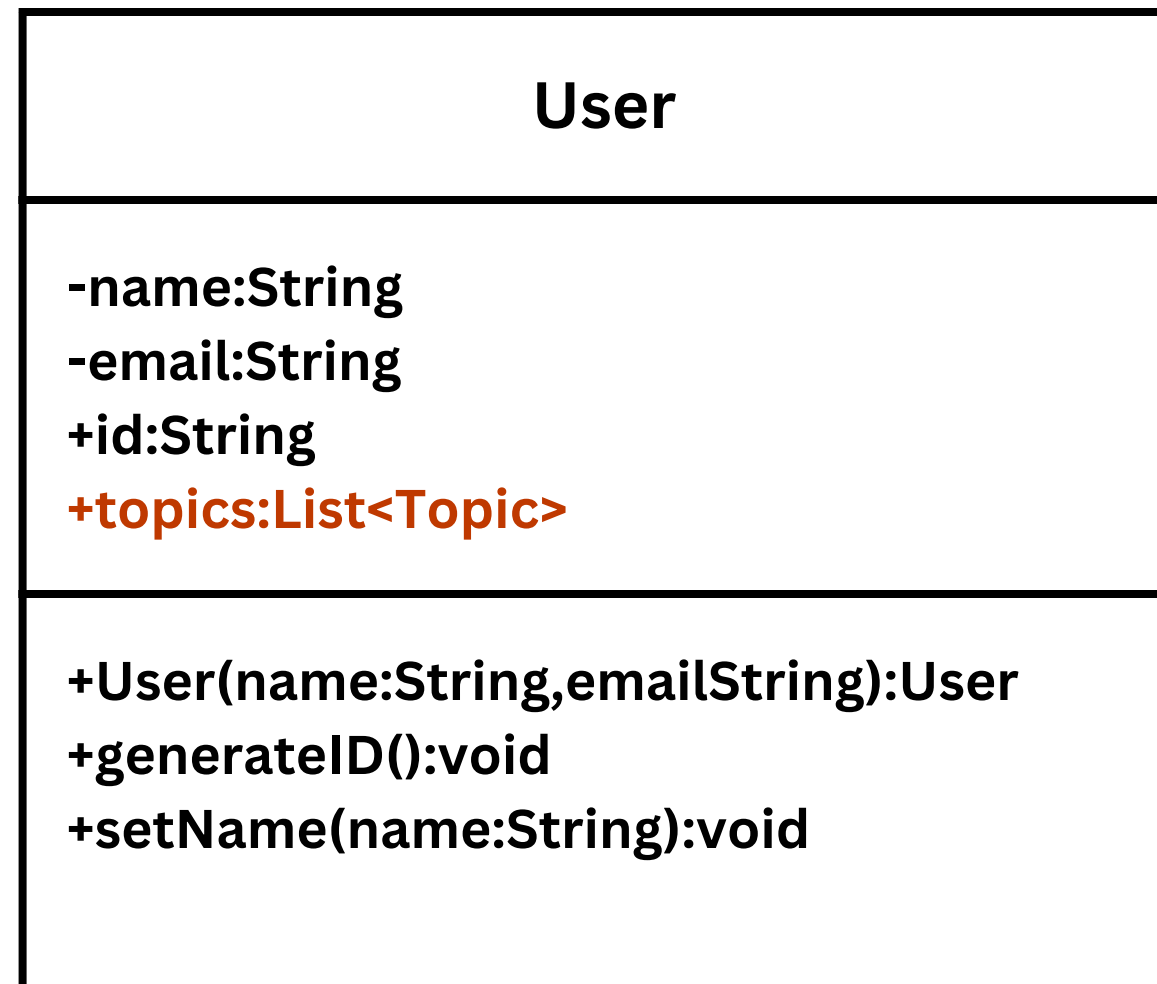
Navigability

A --> B	Class A has access to Class B through: 1- Methods in class A that access attributes in class B, or 2- Attributes in class A that hold instances of class B.
A <-- B	Class B has access to Class A through: 1- Methods in class B that access attributes in class A, or 2- Attributes in class B that hold instances of class A
A --- B	Class A and B have access to attributes and/or instances within each other.





The User class has an attribute topic holding an instance of class Topic.
Therefore, User has access to Topic but not the other way round.



The User class would have a list of Topic objects when implemented to reflect the 0..* association..

UML Class Diagram Modelling Steps

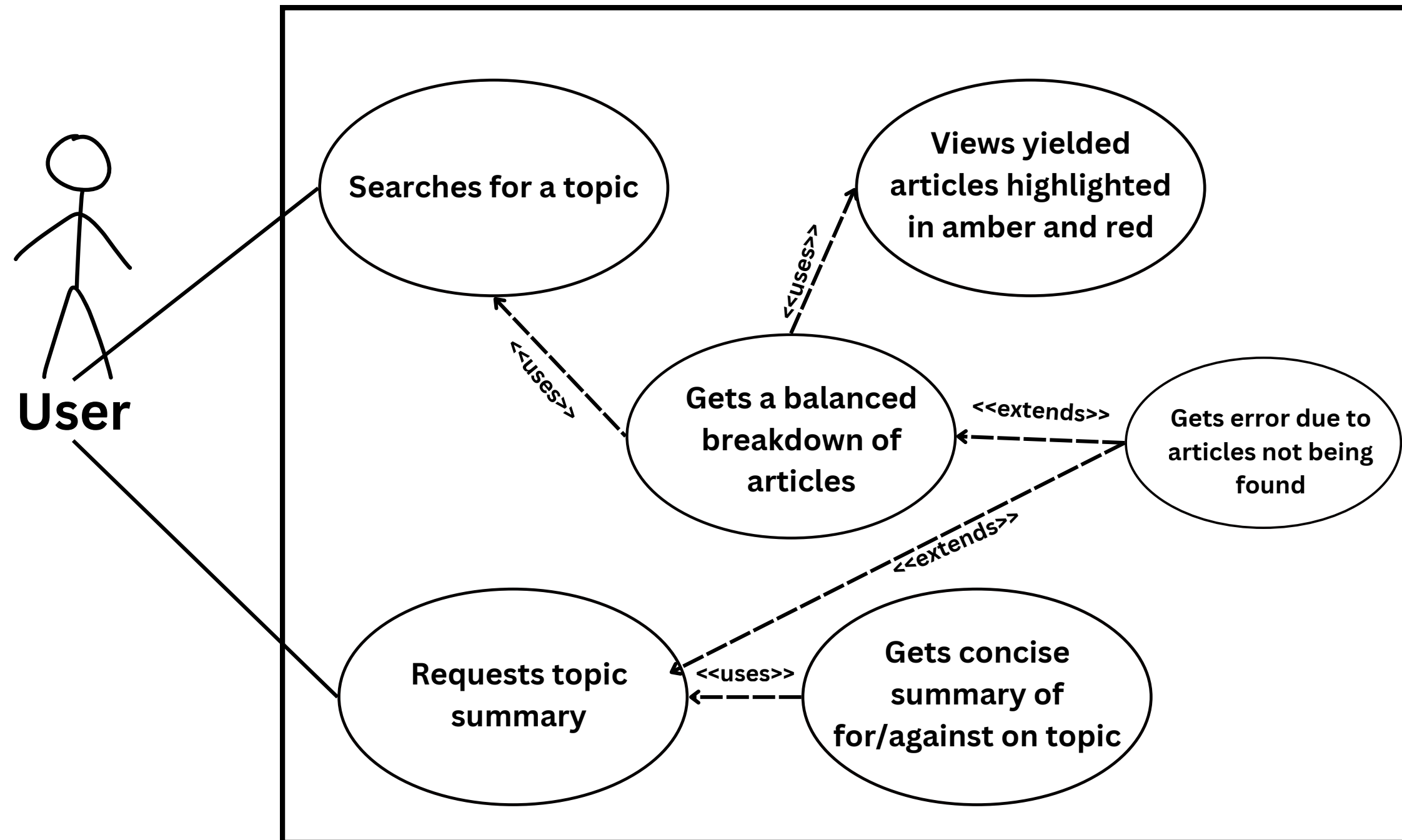
Step 1 -- Identifying Classes and Attributes

<u>Functional Requirement ID</u>	<u>Functional Requirement Description</u>
FR1	The <u>system</u> must display a clear, balanced <u>breakdown</u> of <u>articles</u> providing ‘for’ and ‘against’ <u>perspectives</u> on any <u>news topic</u>
FR2	The system must highlight articles designed to evoke strong <u>emotional reactions</u> , allowing users to be more aware of <u>manipulative content</u> .
FR2.1	The system should have red, amber system to indicate highly triggering and moderately triggering articles.
FR3	The system must provide concise <u>summaries</u> of the <u>key points</u> for/against on specific topics for users who want <u>quick insights</u> without reading full articles
FR4	The system allow <u>users</u> to create <u>accounts</u> and it must store and manage user account <u>information</u> securely.

Steps:

1. Underline all the nouns in the functional requirements table.
2. Categorise them as classes or attributes.
 - a. Can more information be held about this entity? Class.
 - b. Can no more information be held about it? Attribute.

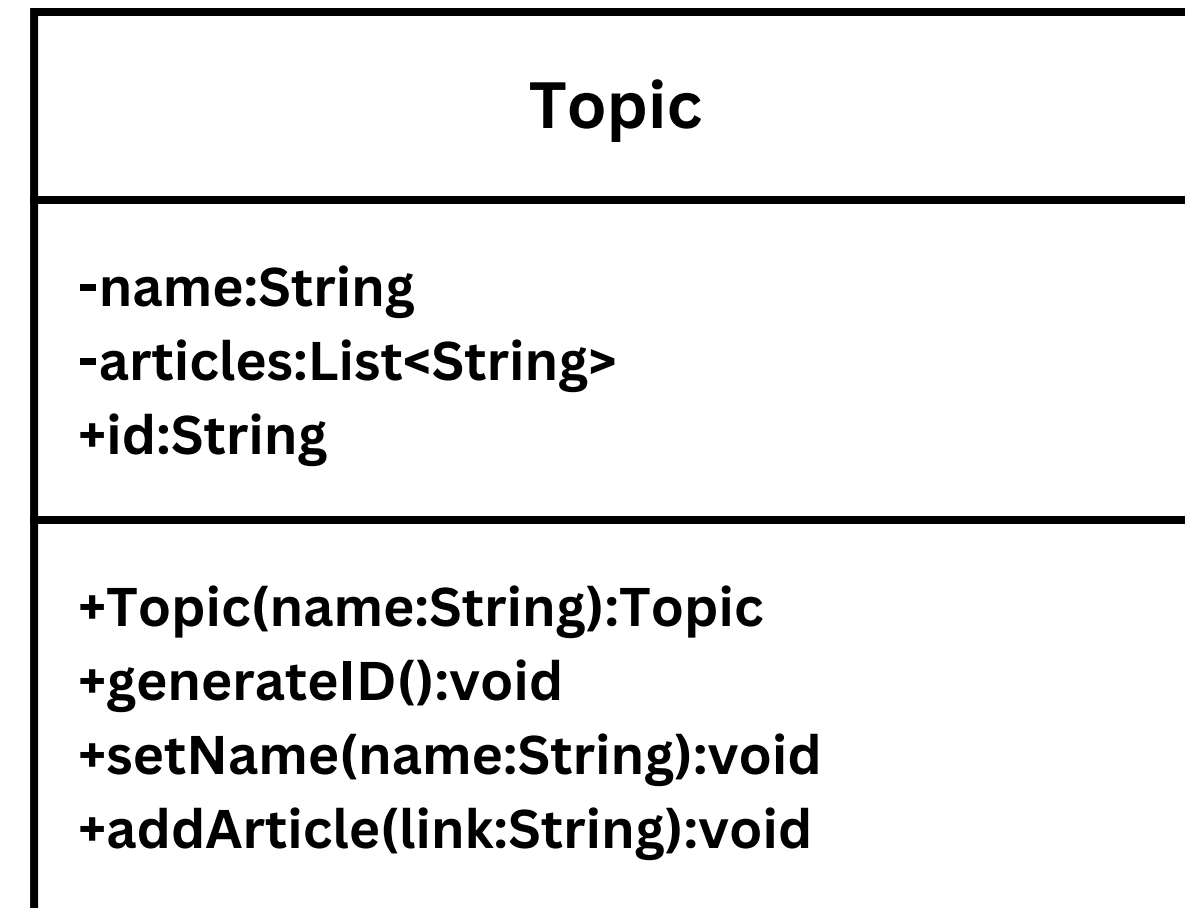
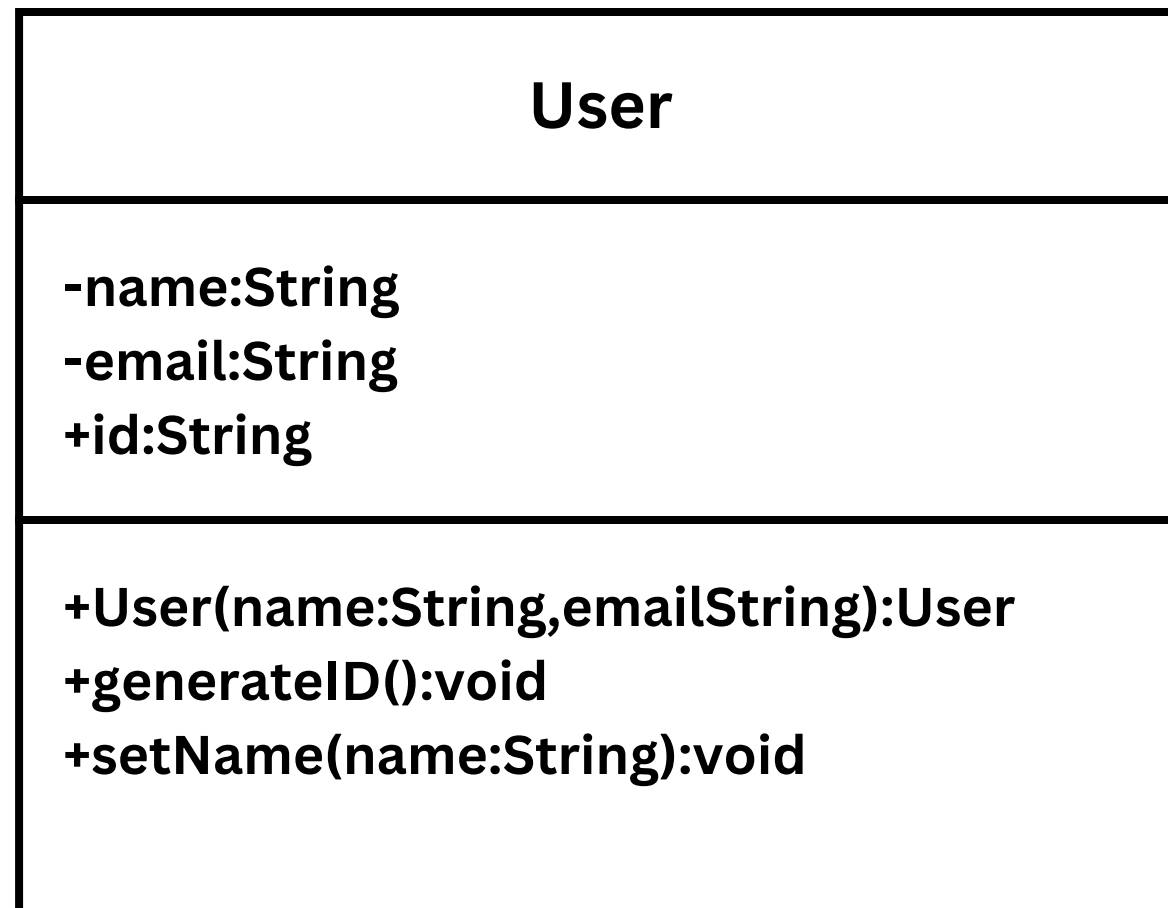
Step 2 -- Identifying Methods



Steps:

1. Underline all the verbs in the use case diagram.
2. Categorise them in the classes where they should be implemented, not where they should be called..

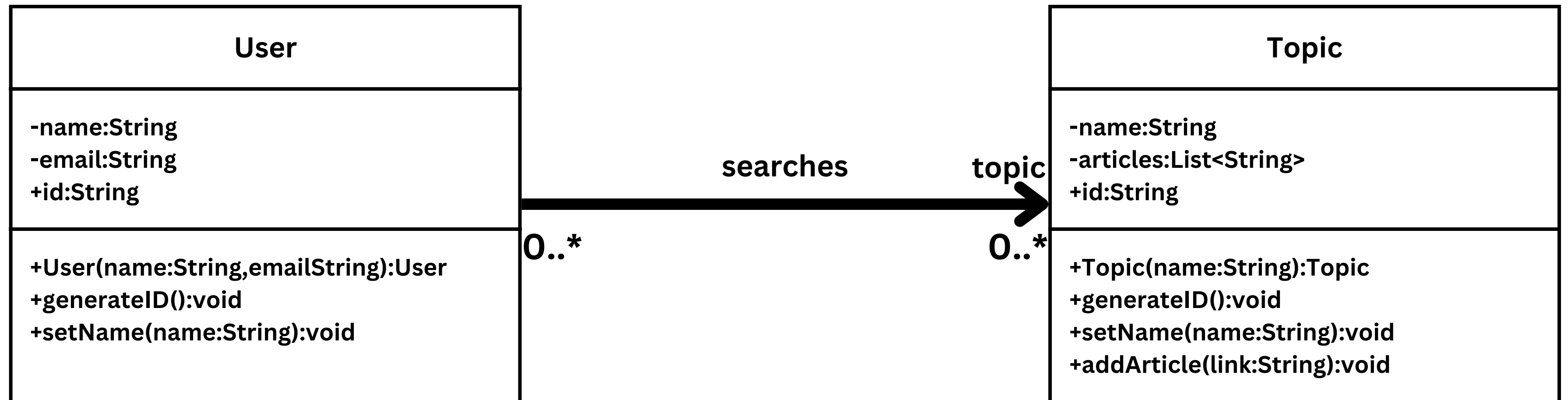
Step 3 -- Drafting Class Diagram



Steps:

1. Create classes based on derived entities.
2. Add methods with their parameters and return types.
3. Add attributes that are needed to fulfil the methods.

Step 4 -- Identifying Associations



Steps:

1. Use functional requirements and use case diagram arrows to identify associations.
2. Add labels on the associations.
3. Add multiplicities on the associations.
4. Add navigabilities on the associations.

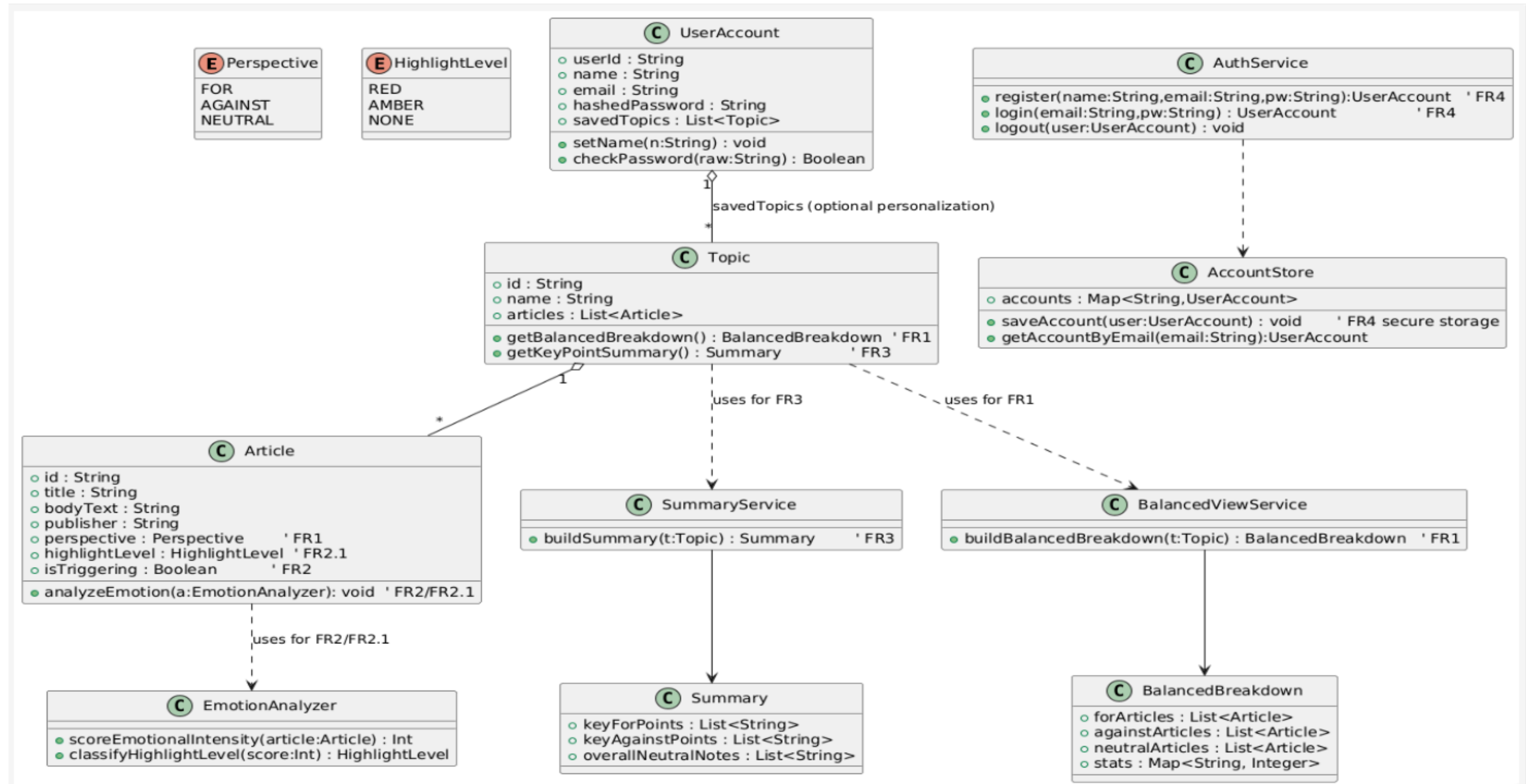
```
public class Topic {  
    // Attributes  
    private String name;  
    private List<String> articles;  
    private String id;  
  
    // Constructor  
    public Topic(String name) {  
        this.name = name;  
        this.articles = new ArrayList<>(); // Initialize the list of articles  
        generateID(); // Generate topic ID  
    }  
  
    // Method to generate a unique ID  
    public void generateID() {  
        this.id = Integer.toHexString(name.hashCode()); // Example ID generation based on name  
    }  
  
    // Method to set the topic's name  
    public void setName(String name) {  
        this.name = name;  
    }  
  
    // Method to add an article to the topic  
    public void addArticle(String article) {  
        articles.add(article); // Add an article link to the topic  
    }  
}
```



```
public class User {  
    // Attributes  
    private String name;  
    private String email;  
    private String id;  
  
    // Association: User can search multiple topics (0..*)  
    private List<Topic> searchedTopics;  
  
    // Constructor  
    public User(String name, String email) {  
        this.name = name;  
        this.email = email;  
        this.searchedTopics = new ArrayList<>(); // Initialize list of searched topics  
        generateID(); // Generate user ID  
    }  
  
    // Method to generate a unique ID  
    public void generateID() {  
        this.id = Integer.toHexString(email.hashCode()); // Example ID generation based  
    }  
  
    // Method to set the user's name  
    public void setName(String name) {  
        this.name = name;  
    }  
}
```

Can AI Replace these steps?

ChatGPT given FRs - provided PlantUML code



Can AI Replace these steps?

What to check for?

- Trace each requirement to a class/method.
- Check cohesion: every class should have one main job.
- Check attributes/methods for consistency with your FRs.
- Check relationships (annotation, direction, multiplicity).
- Make sure naming matches the FRs.
- Tell the story from start to finish. Does the diagram support it?
- Remove duplicate classes.
- Are the notations correct in terms of UML?
- Try with multiple AI tools